

UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Escuela de Ciencias de la Computación

Curso: Base de Datos 2 - Ciclo 2026-2

PROYECTO: MINIGESTOR DE BASE DE DATOS MULTIMODAL

1. Introducción

En este proyecto construirás desde cero un gestor de bases de datos que soporta múltiples tipos de datos: relacionales (tablas), espaciales (coordenadas geográficas), textuales y multimedia (imágenes/audio). El objetivo es comprender cómo funcionan internamente los sistemas de bases de datos modernos, implementando de forma progresiva cada uno de los módulos que componen un motor de base de datos multimodal actual.

El proyecto se desarrolla a lo largo del ciclo y se evalúa mediante entregas parciales. Cada parte se apoya en las estructuras construidas en las partes anteriores, por lo que se recomienda mantener una arquitectura modular desde el inicio.

2. Especificación Funcional

2.1 Parte 1: Base de Datos Relacional (Tablas y SQL)

2.1.1 Gestión de Archivos y Almacenamiento

Implementa diferentes formas de almacenar datos en disco:

- **Heap File:** almacena los registros en páginas de disco en orden de llegada, se debe considerar estrategia de reutilización de espacios libres.
- **Archivo Secuencial Paginado:** mantén los registros ordenados por una clave. Implementa:
 - Inserción manteniendo el orden
 - Eliminación lazy
 - Estrategia de reorganización (por ejemplo, cuando haya más del 30% de espacio desperdiciado)

2.1.2 Estructuras de Indexación y optimización

Implementa índices para acelerar las búsquedas:

- Índice B+ agrupado
- Índice B+ no agrupado
- Índice Hash Dinámico (Extendible Hashing)
- External Algorithms: Implementa `ORDER BY` mediante *External Sorting* (k-way merge), así como `GROUP BY` y `JOIN` optimizados con *External Hashing* o el uso estratégico de índices.

2.1.3 Procesamiento de Consultas SQL

Implementa un parser SQL que permita al usuario interactuar con la BD mediante consultas básicas:

```
SELECT [*] FROM tabla WHERE condicion
```

```
SELECT [*] FROM tabla ORDER BY | GROUP BY ...  
INSERT INTO tabla VALUES (...)  
DELETE FROM tabla WHERE condicion
```

No es necesario implementar todo el estándar SQL, solo lo esencial que de soporte a las técnicas implementadas.

2.1.4 Transacciones y Concurrency

Implementa mecanismos que permitan a múltiples usuarios acceder a la BD simultáneamente de forma segura:

- **Transacciones:** soporta BEGIN TRANSACTION y END TRANSACTION para agrupar operaciones.
- **Control de Concurrency:** implementa algún mecanismo de bloqueo (locks) o control de concurrency.
- **Demostración obligatoria:** crea una simulación con hilos (threads) donde se vea:
 - Múltiples transacciones ejecutándose simultáneamente
 - Situaciones de race condition (competencia por recursos)
 - Cómo tu sistema las maneja correctamente

2.1.5 Interfaz de Usuario (Frontend)

Crea una interfaz gráfica amigable que incluya 4 paneles principales:

- **Panel de Archivos:** muestra las tablas cargadas y su estructura.
- **Panel de Consultas:** editor donde el usuario escribe consultas SQL.
- **Panel de Resultados:** tabla que muestra los resultados de las consultas.
- **Panel de Plan de Ejecución:** visualización de cómo se ejecutó la consulta (qué índices se usaron, orden de operaciones, etc.).

2.1.6 Comparación Experimental de Técnicas

Realiza un análisis comparativo entre las diferentes técnicas implementadas.

Gestión de Archivos: compara Heap File vs Archivo Secuencial Paginado midiendo tiempo de inserción (1 000, 10 000, 100 000 registros), tiempo de búsqueda por clave primaria, espacio en disco utilizado y tiempo de reorganización. Identifica en qué casos es mejor cada técnica.

Estructuras de Indexación: compara B+ agrupado vs B+ no agrupado vs Hash Dinámico evaluando búsquedas por igualdad exacta, búsquedas por rango y ordenamiento. Mide tiempo de construcción del índice, tiempo de consulta, espacio adicional requerido y rendimiento con inserciones/eliminaciones frecuentes.

Presentación: incluye gráficas comparativas, tabla resumen con ventajas/desventajas de cada técnica, y conclusiones sobre cuándo usar cada estructura.

2.2 Parte 2: Base de Datos Espacial (Coordenadas y Mapas)

2.2.1 Implementación

Implementa un índice **R-Tree** para manejar datos geográficos (ubicaciones de tiendas, rutas, zonas, etc.) con puntos en 2D (latitud, longitud). Tu sistema debe soportar:

- Consultas por rango (ej.: todas las tiendas en un radio de 5 km).
- k-NN para encontrar los k vecinos más cercanos (ej.: las 10 gasolineras más cercanas).
- Intersección con polígonos (ej.: sucursales dentro de un distrito).

Implementa ambas métricas de distancia: Euclidiana y Geodésica (Haversine).

2.2.2 Visualización

Agrega al frontend un panel de mapa que visualice los puntos sobre un mapa interactivo (Leaflet, Google Maps API, etc.) y muestre los resultados de las búsquedas espaciales resaltados en el mapa.

2.2.3 Extensión SQL

Extiende tu parser SQL para soportar consultas espaciales:

```
-- Ejemplo de consultas que deberías soportar:  
SELECT * FROM tiendas WHERE distancia(ubicacion, POINT(-12.0464, -77.0428)) < 5000;  
SELECT * FROM restaurantes ORDER BY distancia(ubicacion, mi_ubicacion) LIMIT 10;
```

2.2.4 Comparación Experimental de Técnicas

Compara Búsqueda Secuencial vs R-Tree (tu implementación) vs GIST de PostgreSQL. Evalúa con consultas por rango (radio 1 km, 5 km, 10 km), consultas k-NN (k = 10, 50, 100), y datasets de 1 000, 10 000 y 100 000 puntos. Mide tiempo de construcción del índice, tiempo de consulta (promedio de 100 consultas) y uso de memoria/espacio en disco. Presenta gráficas comparativas de rendimiento y tabla resumen indicando cuándo usar cada técnica.

2.3 Parte 3: Base de Datos Vectorial — Búsqueda de Texto (Full-Text Search)

2.3.1 Implementación

Implementa un sistema de búsqueda en documentos de texto utilizando **SPIMI (Single-Pass In-Memory Indexing)** para el índice invertido. Implementa dos técnicas de ranking: **TF-IDF + Similitud del Coseno** y **BM25**.

2.3.2 Comparación Experimental de Técnicas

Compara TF-IDF + Coseno vs BM25 vs GIN de PostgreSQL. Ejecuta las mismas consultas con las tres técnicas usando datasets de 1 000, 10 000 y 100 000 documentos, y consultas de diferente longitud (1 palabra, 3 palabras, 5+ palabras). Mide tiempo de construcción del índice, tiempo de consulta, relevancia (Precision@10, Recall@10) y uso de memoria/espacio en disco. Presenta gráficas comparativas de rendimiento y tabla con ventajas/desventajas de cada técnica.

2.3.3 Extensión SQL

Extiende tu parser para búsquedas de texto:

```
-- Ejemplos de consultas:  
SELECT * FROM documentos WHERE MATCH(contenido, 'base de datos vectorial')  
    USING TF_IDF LIMIT 10;
```

```
SELECT *, SCORE() as relevancia FROM articulos
WHERE MATCH(texto, 'machine learning')
USING BM25
ORDER BY relevancia DESC;
```

2.4 Parte 4: Base de Datos Vectorial — Multimedia (Imágenes y Audio)

2.4.1 Extracción de Características (Feature Extraction)

Implementa una función **ExtractFeatures(objeto)** que convierte imágenes/audio en vectores numéricos para buscar contenido multimedia similar basándose en sus características. Pipeline de procesamiento:

- **Extracción de Patrones Locales:** para imágenes usa **SIFT** (Scale-Invariant Feature Transform); para audio usa **MFCC** (Mel-Frequency Cepstral Coefficients).
- **Cuantización (Bag of Visual/Audio Words):** aplica **K-Means** o **Tree Quantization** sobre los descriptores para obtener K centroides (palabras visuales/auditivas).
- **Creación del Histograma:** cuenta la frecuencia de cada "palabra", aplica **TF-IDF** y obtiene un vector de dimensión K.

2.4.2 Índices y Métricas

Implementa índices especializados para vectores de alta dimensionalidad: **IVF (Inverted File Index)** y **HNSW (Hierarchical Navigable Small World)**. Implementa tres métricas de similitud: Distancia Euclidiana, Producto Punto y Similitud del Coseno.

2.4.3 Extensión SQL

Agrega sintaxis para búsquedas por similitud:

```
-- Búsqueda de imágenes similares
SELECT * FROM imagenes
WHERE SIMILAR_TO('foto_consulta.jpg', k=10)
USING HNSW WITH METRIC=cosine;

-- Búsqueda de canciones similares
SELECT nombre, artista, SIMILARITY_SCORE() as score
FROM canciones
WHERE SIMILAR_TO('audio_query.mp3', k=5)
USING IVF WITH METRIC=euclidean
ORDER BY score DESC;
```

2.4.4 Comparación Experimental de Técnicas

Compara IVF vs HNSW, y Distancia Euclidiana vs Similitud del Coseno. Evalúa con consultas k-NN (k=10) usando datasets de 1 000, 10 000 y 100 000 imágenes/audios. Mide tiempo de construcción del índice, tiempo de consulta, precisión (Recall@10) y uso de memoria. Presenta gráficas de rendimiento (tiempo vs. tamaño del dataset), galería visual mostrando casos de éxito y error, y tabla comparativa con recomendaciones.

2.5 Parte 5: Aplicación de Inteligencia Artificial

Construye una aplicación real que utilice las capacidades multimodales de tu base de datos:

- La aplicación debe consumir el **API REST/GraphQL** de tu motor de base de datos.

- Debe integrar al menos **2 tipos de datos** (relacional, espacial, texto o multimedia).
- Implementa una interfaz de usuario que demuestre las capacidades del sistema.
- Elige una de las opciones de aplicación descritas en el **Anexo A**.

3. Entregables

- **Código fuente** en repositorio Git (GitHub/GitLab).
- **Documentación técnica (README):** arquitectura del sistema, arquetipo/organización del código fuente, manual de instalación.
- **Video demo** (5–10 minutos) mostrando todas las funcionalidades.
- **Informe incremental** (diseño arquitectónico, dominio de datos, explicación de algoritmos; incluye la parte experimental).
- **Presentación final** (15 minutos + 5 min de preguntas).

4. Fechas Importantes

Hito	Semana	Entregable
Avance 1	Semana 6	Parte 1 completa
Entrega Parcial	Semana 8	Partes 1 y 2 completas
Avance 3	Semana 12	Partes 3 y 4 completas
Entrega Final	Semana 15	Todo completo + documentación
Presentaciones	Semana 16	Exposición de proyectos

Anexo A: Opciones de Aplicación (Parte 5)

Opción A: RAG para Documentos Académicos

RAG = Retrieval Augmented Generation (Generación Aumentada por Recuperación).

¿Qué hace? Un chatbot que responde preguntas basándose en una colección de papers académicos.

Componentes:

- **Ingestión de Documentos:** extrae texto de archivos PDF, divide cada documento en chunks (fragmentos de 200-500 palabras) y guarda la metadata (autor, año, título, abstract) en tablas relacionales.
- **Indexación:** indexa los chunks usando el índice de texto (Parte 3). Opcionalmente, genera embeddings y usa índices vectoriales (Parte 4).
- **Sistema de Consulta:** el usuario hace una pregunta, el sistema busca los chunks más relevantes, los envía junto con la pregunta a un LLM (GPT, Claude, etc.), y el LLM genera una respuesta basada en los documentos recuperados.

Ejemplo de uso:

Usuario: "¿Cuáles son las ventajas de HNSW sobre IVF?"

Sistema: [Busca en papers] → [Encuentra 3 chunks relevantes] → [Envía a LLM]

LLM: "Según García et al. (2023), HNSW ofrece mejor precisión..."

Opción B: Sistema de Recomendación E-commerce

¿Qué hace? Tienda en línea que recomienda productos combinando múltiples estrategias.

Componentes:

- **Base de Datos:** productos (nombre, precio, categoría, descripción) en tablas relacionales; imágenes de productos en índices multimedia; ubicación de tiendas en índices espaciales.
- **Sistema de Recomendación Híbrido:** por metadata ("usuarios que compraron X también compraron Y"), por contenido (productos visualmente similares), por texto (descripciones similares), y fusión de los scores de los diferentes métodos.

Ejemplo de recomendación:

Usuario ve: "Laptop HP 15"

Sistema busca:

- Productos de categoría similar (metadata)
- Laptops con imágenes similares (visual)
- Productos con descripciones parecidas (texto)

Fusión de resultados → Top 10 recomendaciones

Opción C: Detección de Copyright en Audio

¿Qué hace? Detecta si una canción contiene fragmentos de otras canciones (como Content ID de YouTube).

Componentes:

- **Base de Datos de Canciones Originales:** descarga canciones de Spotify (o usa un dataset público), extrae características con MFCC (Parte 4) e indexa en la base de datos vectorial.
- **Sistema de Detección:** el usuario sube una canción (posible remix/cover); el sistema la divide en segmentos de 5-10 segundos y, para cada segmento, busca los k audios más similares en la BD. Si encuentra coincidencias fuertes, reporta un posible caso de copyright.
- **Visualización:** timeline de la canción subida, marcando los segmentos que coinciden con canciones originales y el porcentaje de similitud.

Ejemplo:

```
Canción subida: "remix_2024.mp3"
Detección:
- 0:00-0:15 → 92% similar a "Song A" de Artist X
- 0:45-1:10 → 87% similar a "Song B" de Artist Y
- Conclusión: Posible uso no autorizado detectado
```

Opción D: Reconocimiento Facial

¿Qué hace? Aplicación móvil (o con Raspberry Pi) que reconoce rostros en tiempo real.

Componentes:

- **Base de Datos de Rostros:** usa un dataset público (LFW, CelebA, etc.), agrega fotos propias y de tus compañeros, extrae embeddings faciales con modelos preentrenados (FaceNet, ArcFace) e indexa en HNSW o IVF.
- **Aplicación Móvil o Sistema Embebido:** captura foto/video de la cámara, detecta rostros (OpenCV o MTCNN), extrae el embedding del rostro detectado, busca en la BD el rostro más similar y muestra el nombre de la persona (si está registrada).
- **Enfoque en Eficiencia:** optimiza para búsquedas en menos de 100 ms; si usas Raspberry Pi, considera índices más ligeros; implementa cache (Redis) para personas frecuentes.

Ejemplo de uso:

```
Cámara detecta rostro → Extrae embedding → Busca en BD (5ms)
→ Match encontrado: "Juan Pérez" (95% confianza)
→ Muestra nombre en pantalla
```

Datasets Recomendados

- **Para RAG:** ArXiv papers, papers de conferencias (NeurIPS, ICML, etc.), su propio material académico.
- **Para E-commerce:** Amazon Product Dataset, Flipkart Dataset.
- **Para Audio:** Million Song Dataset, Free Music Archive.
- **Para Reconocimiento Facial:** LFW, CelebA, VGGFace2.